

University Number: _____

THE UNIVERSITY OF HONG KONG

**SCHOOL OF COMPUTING AND DATA SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**

COMP7104/DASC7104 - Advanced Database Systems

Date: May 21, 2025

Time: 6:30pm-8:30pm

<INSTRUCTION ABOUT NUMBER OF QUESTION TO BE ANSWERED: OVERALL
AND FROM EACH SECTION>

23 questions overall, divided into two sections as follows:

Section 1 – Multiple-choice questions – 15 questions

Section 2 – Detailed-answer questions – 8 questions

<INDICATION OF COMPULSORY QUESTIONS>

None.

<INFORMATION ABOUT THE MARK VALUE OF QUESTIONS>

Section 1 – 2 points per question – 30 / 100 points in total

Section 2 – number of points depends on the question – 70 / 100 points in total

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

Candidates are permitted to bring one sheet of A4-sized paper with printed or written notes on both sides to the examination.

Mark your University Number on each page (top right corner).

University Number: _____

Brand and Type of Calculator: _____

General indications

- For questions involving arithmetic expressions, numerical work or derivations, credit will be given only if the work leading to the answer is shown.

Section 1 – Multiple-choice questions – 15 questions – 2 points per question

Circle the true answer(s) to each question. Each question may have zero, one, or several valid answers. For each question, only the identification of all and only the valid answers will allow you to obtain the assigned points.

Q1.1 Which of the following assertions about join algorithms are true?

- (A) Block (a.k.a. Chunk) Nested Loops Join will always perform at least as well as Simple (Page) Nested Loops Join in terms of I/O cost.
- (B) For Nested Loop Join, to minimize I/O cost, it is best to make the smaller relation the outer branch of the loop.
- (C) If we are joining two tables that are very different in size, using Index Nested Loop Join, and both relations have indexes on their join column, to minimize I/O cost, it is best to query the index of the smaller relation.

Q1.2 Suppose we do a Simple (Page) Nested Loop Join on two tables, P and Q. The same amount of pages are required to store P and Q, but Q has three times more records per page than P, on average. Which one of the following options is true ?

- (A) $P \bowtie Q$ does fewer I/O operations than $Q \bowtie P$.
- (B) $Q \bowtie P$ does fewer I/O operations than $P \bowtie Q$.
- (C) $P \bowtie Q$ does the same number of I/O operations as $Q \bowtie P$.

Q1.3 Which of the following assertions about join algorithms are true?

- (A) If both tables in a simple nested loop join fit entirely in main-memory, the order of inner and outer tables does not significantly affect I/O cost.
- (B) If neither table fits entirely in main-memory, I/O cost would be lower if we process both tables on a per-block basis rather than per-tuple basis.
- (C) If the smaller table fits in main-memory, while the largest one does not fit, we should use a hash-based join instead of a sort based one.

Q1.4 A nested loop join will output a sorted stream of tuples under which of the following conditions?

- (A) All nested loop joins will output a sorted stream.
- (B) Outer table is already sorted.
- (C) Inner table is already sorted.

Q1.5 Consider the following join algorithm. Given an equijoin on tables R and S (with $[R] < [S]$), but no indexes on the join columns initially, the algorithm will take the following steps in sequence (1) bulk-load an Alternative-1 B+-tree on R, (2) perform an Index Nested Loops Join on S JOIN R using the index, and (3) delete the index on R. Assuming B (buffer size) $< [R] < [S] < (B-1)^2$ mark which statements are true.

- (A) This algorithm generates more I/Os than Grace Hash Join (without recursive partitioning).
- (B) This algorithm generates more I/Os than a 2-pass Sort-Merge join.
- (C) This algorithm generates more random I/Os than Block Nested Loops Join.

Q1.6 Which join operators among the following ones are not pipeline breakers in query execution plans?

- (A) Index Nested Loop Join.
- (B) Chunk Nested Loop Join.
- (C) Grace-Hash Join.

Q1.7 Which of the following assertions about the System R / Selinger algorithm for query optimization are true?

- (A) The resulting plan may include a cartesian product.
- (B) The resulting plan is always a pipelined one.
- (C) It only keeps the cheapest sub-plans as it proceeds from one stage to the next.

Q1.8 Which statements about the System R / Selinger algorithm for query optimization are true?

- (A) It always guarantees the generation of an optimal execution plan by exhaustively evaluating all possible plans to ensure the lowest cost for query execution.
- (B) Predicate (selection) and projection pushdown will always improve performance.
- (C) It has a search algorithm that builds plans in top-down manner.

Q1.9 Which statements about histograms for cost and size estimates in the System R / Selinger algorithm for query optimization are true?

- (A) Equi-width histograms use the same range size for each bucket.
- (B) Equi-depth histograms vary the range size of buckets so that the total number of occurrences for each bucket is roughly the same.
- (C) Unless we work with the assumption of uniform distribution within buckets, we can only use histograms for equality or range conditions that fall exactly on range boundaries.

Q1.10 Which statements about locking protocols and potential issues in concurrency control are true?

- (A) Cascading aborts are possible under Strict Two-Phase Locking (Strict 2PL).
- (B) 2PL guarantees a conflict-serializable schedule.
- (C) For a schedule following strong strict 2PL, the dependency graph is guaranteed to be acyclic.

Q1.11 Which statements about concurrency control and concurrent schedules are true?

- (A) Dirty reads may occur in conflict-serializable schedules.
- (B) Lost updates may occur in conflict-serializable schedules.
- (C) Cascading aborts may occur in conflict serializable schedules.

Q1.12 Which statements about parallel database architectures are true ?

- (A) In shared-disk architectures compute and storage can scale independently.
- (B) Shared-memory architectures are the closest to a centralized architecture.
- (C) In shared-nothing architectures, scaling the storage layer is easier than in shared-disk ones.

Q1.13 Which of the following assertions are true about data partitioning & query evaluation in PDBMSs?

- (A) If we partition data using the round robin scheme, insertion by key will incur a higher cost than with range or hash partitioning.
- (B) A join over two tables can benefit more from a round-robin partitioning of their records than from a hash partitioning on the join attribute.
- (C) A join over two tables can benefit more from a range partitioning on the join attribute than from a round-robin partitioning of their records.

Q1.14 Which of the following are reasons to replicate data in NoSQL databases ?

- (A) To allow multiple nodes to split up the work for a large volume of requests.
- (B) To reduce latency by allowing clients to fetch data from a nearby server.
- (C). To handle ever-increasing database sizes.

Q1.15 Which of the following assertions are true for NoSQL database systems ?

- (A) They were designed to cope with high rates of simple read / write operations.
- (B) They rely on replication for data scalability.
- (C) They were designed for transactional workloads.

Section 2 – Detailed-answer questions – 8 questions – 70 points in total

Q2.1 (2 points) – Joins - Assume we want to perform a join between two tables R and S using hashing. Assume that none of the tables is small enough so that its hash table (denoted HT_R and HT_S respectively) would fit in main memory, and that a large number of distinct values of the join attribute hash to the same bucket using a first hash function h_p . Would creating hash tables for R and S using h_p and then

rehashing into another hash table using some hash function $h \neq h_p$ for large h_p buckets work ? Explain why or why not?

Q2.2 (4 points) Consider two relations R and S to be joined on their common attribute 'a' by $R.a = S.a$

- R has 40,000 and each record is 10 bytes long.
- S has 500 tuples, and each record is 50 bytes long.
- Page size is 1KB.
- The buffer pool is 50 pages large.
- The fill factor for all hash tables is 0.7.
- There are no indexes.

Considering all the join algorithms covered so far in this class, what is the minimum estimated I/O cost to execute this join? Exclude the final write I/O cost (saving the join result) from your solution.

Q2.3 (5 points) - Cardinality estimation - Consider a table Team(Id, name, city, country). You need to estimate the cardinality of the following query:

```
SELECT * FROM Team WHERE country= 'UK' AND city = 'London'
```

For the following questions, assume Team has 10,000 rows with 5% having country 'UK', and 0.3% having city = 'London'.

(a) (2.5 points) Under a uniform data assumption and independent predicates assumption, what is the estimated cardinality of this query?

(b) (2.5 points) Is the result from previous question an overestimate or underestimate of the true cardinality?

Q2.4 (11 points) - Serializability - Consider the schedule consisting of four transactions in the next table, where R and W stand for ‘Read’ and ‘Write’, respectively, and time flows from left to right.

Time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
T ₁					R(A)			R(B)	W(B)	
T ₂		R(E)	W(E)				R(D)	W(C)		W(A)
T ₃	R(B)		W(C)			W(A)				
T ₄			R(B)	R(D)	W(D)					

(a) (2 points) Is this schedule serial (explain) ?

(b) (5 points) Compute the conflict dependency graph for this schedule.

(c) (2 points) Is this schedule conflict-serializable (explain) ?

(d) (2 points) Is this schedule possible under regular (not strict) 2PL (explain)?

Q2.5 (12 points) – Deadlock avoidance - Consider the following lock requests of four transactions, where S stands for shared lock and X stands for exclusive lock. Assume that transactions never release a granted lock. Assume that in terms of priority $T_1 > T_2 > T_3 > T_4$ (e.g., T_1 is older than T_2 , etc).

(a) (6 points) Wait-Die. Complete the table, determining at each time step (time flows left to right) if the lock request is granted ('g'), blocked ('b'), aborted ('a'), or already dead ('-'), under the following policies.

time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈
T ₁		X(A)						
T ₂				X(C)				
T ₃	X(A)		S(C)		X(B)			X(D)
T ₄						S(B)	X(C)	
LM	g							

(b) (6 points) Wound-Wait. Complete the table, determining at each time step if the lock request is granted ('g'), blocked ('b'), killed (granted by aborting another transaction) ('k'), or the requester is already dead ('-').

time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈
T ₁		X(A)						
T ₂				X(C)				
T ₃	X(A)		S(C)		X(B)			X(D)
T ₄						S(B)	X(C)	
LM	g							

Q2.6 (5 points) – Concurrency control – Consider the following schedule, involving two transactions, where times flows from left to right.

T1: W(X) W(Y)
T2: R(X) R(Y)

(a) (2.5 points) Is this schedule conflict serializable (explain) ?

(b) (2.5 points) Is this possible under regular (not strict) 2PL ?

Q2.7 (18 points) - Selectivity Estimation - Consider a relation $R(a, b, c)$ with 5000 tuples. We have an index on 'a' with 50 unique values in the range $[1, 50]$ and an index on 'b' with 100 unique values in the range $[1, 100]$. We do not have an index on 'c'.

Use selectivity estimation to estimate the number of tuples produced by the following queries.

(a) (1.5 points) `SELECT * FROM R`

(b) (1.5 points) `SELECT * FROM R WHERE a=40`

(c) (1.5 points) `SELECT * FROM R WHERE b=40`

(d) (1.5 points) SELECT * FROM R WHERE c=40

(e) (1.5 points) SELECT * FROM R WHERE a <= 10

(f) (1.5 points) SELECT * FROM R WHERE b <= 20

(g) (1.5 points) SELECT * FROM R WHERE c <= 25

(h) (1.5 points) SELECT * FROM R WHERE a<=10 AND b<=20

(i) (1.5 points) SELECT * FROM R WHERE $a \leq 10$ AND $c \leq 25$

(j) (1.5 points) SELECT * FROM R WHERE $a \leq 10$ OR $b \leq 20$

(k) (1.5 points) SELECT * FROM R WHERE $a = b$

(l) (1.5 points) SELECT * FROM R WHERE $a = c$

Q2.8 (13 points) PDBMS - Distributed Query Evaluation – Consider the following schema:

- Player(Player_Id, name) – Primary Key Player_Id – record size $6 = 2 + 4$ bytes
- PlayerCoach(Player_Id, Coach_Id, Nb_years) – Primary Key (Player_Id, Coach_Id) – record size $6 = 2 + 2 + 2$ bytes
- Coach(Coach_Id, name, Division_Id) – Primary Key Coach_Id – record size $8 = 2 + 4 + 2$ bytes
- Division(Division_Id, name) – Primary Key Division_Id – record size $6 = 2 + 4$ bytes

For performance reasons, these tables may be partitioned and / or replicated across nodes, such that all nodes can do a similar amount of work in parallel.

We anticipate four-way join queries like the following:

```
SELECT *
FROM Player P JOIN PlayerCoach PC ON P.Player_Id = PC.Player_Id
      JOIN Coach C ON PC.Coach_Id = C.Coach_Id
      JOIN Division D ON D.Division_Id = C.Division_Id
WHERE P.name LIKE 'A%'
```

for any 'A' initial letter.

- Pages are 4KB in size.
- There are 4 nodes in the system.
- The Player table contains 40,000 records.
- The PlayerCoach table contains 100,000 records.
- The Coach table contains 100 records.
- The Division table contains 25 records.

(a) (6 points) Assuming that initially all the data is located at a single node (any of the four nodes), which data partitioning and / or replication strategy minimizes the *total network data transfer* for the given query, including the initial costs for replication and or partitioning? Assuming final join results do not need to be collected at one central node, give the cost of this strategy in terms of number of pages transferred across the network.

(b) (4 points) Assuming now the selected strategy from (a) is followed and that the usual reduction factor of the LIKE condition is 1/20, what is the estimated amount of data (join results) to be transferred back to a central node.

University Number: _____

(c) (3 points) Assume now the following query must be computed instead, involving grouping and Max aggregation, with each of the four nodes computing as much as possible locally, including a partial aggregation, before sending their result to one central node among them for the final aggregation. What is the amount of data transferred over the network in this case?

```
SELECT D.name, Max(PC.Nb_Years)
FROM Player P JOIN PlayerCoach PC ON P.Player_Id = PC.Player_Id
      JOIN Coach C ON PC.Coach_Id = C.Coach_Id
      JOIN Division D ON D.Division_Id = C.Division_Id
WHERE P.name LIKE 'A%'
GROUP BY D.name
```

END OF PAPER